R Lab6. Estructura de proteína con R

Bioinformática. Ingeniería Biomédica

Las proteínas, a diferencia de los nucleótidos, muestran una gran variedad de conformaciones tridimensionales necesarias para sus diversos roles estructurales y funcionales. Los aminoácidos en una proteína van a determinar su estructura tridimensional, por lo que la frecuencia de algunos aminoácidos o combinaciones de ellos en la secuencia juega un papel importante en la determinación de las estructuras secundarias de la proteína: hélices α, láminas β o hélices de colágeno.

Una tarea importante será predecir la estructura secundaria de un péptido, que algunas veces es un poco más complicado que evaluar la frecuencia y posición de determinados aminoácidos.

Por otra parte, ciertas propiedades físicoquímicas de una proteína también estan determinadas por las propiedades de los aminoácidos que conforman su secuencia. Por ejemplo, la acidez se puede decidir en función de la fracción de aminoácidos ácidos en la proteína.

Todas las propiedades de las proteínas, desde la estructura hasta las fisicoquímicas, son fundamentales para determinar su función biológica.

En este laboratorio vamos a trabajar en R con proteínas, en sus niveles de secuencia y estructura.

El paquete `Bio3D`

En realidad el paquete `Bio3D` es un conjunto de paquetes de R que contiene utilidades para el análisis de datos de estructura, secuencia y trayectoria biomolecular, aprovechando las capacidades gráficas y estadísticas del entorno R y, siendo muy útil para el análisis exploratorio de datos estructurales.

La funcionalidad para varias tareas de análisis complejas está empaquetada en paquetes separados, que incluyen:

- `bio3d` que es el paquete básico, que incluye alineación, comparaciones de secuencia y estructura, y análisis inter-conformacional con PCA.
- `bio3d.nma` dirigido a predecir y contrastar dinámicas funcionales entre familias de proteínas.
- `bio3d.cna` para análisis de estructura de proteínas y redes de correlación para caracterizar movimientos proteicos correlacionados subyacentes a la regulación alostérica.
- `bio3d.web` para realizar análisis interactivos en línea de estructuras de proteínas y su dinámica.
- `bio3d.eddm` para caracterizar cambios conformacionales significativos funcionalmente.
- `bio3d.view` para visualización interactiva en 3D.

En este laboratorio no es posible explorar todas las opciones, por lo que haremos un repaso por las cuestiones más básicas.

Instala y carga el paquete `bio3d` disponible en CRAN con todas sus dependencias (argumento `dependencies = TRUE` de `install.packages()`)

Solución:

```
if(!("bio3d" %in% installed.packages())
  install.packages("bio3d", dependencies=TRUE)

library("bio3d")
```

Trabajar con ficheros PDB

Los archivos del Banco de Datos de Proteínas (PDB, *Protein Data Bank*) son el formato más común para la distribución y almacenamiento de datos de coordenadas biomoleculares de alta resolución. En su forma más básica, los archivos de coordenadas PDB contienen una lista de todos los átomos de una o más estructuras moleculares. La posición de cada átomo está definida por sus coordenadas x, y, z en un sistema de coordenadas ortogonal convencional. Además, los archivos PDB comúnmente (pero no siempre) incluyen datos adicionales, como listas de elementos de estructura secundaria observados. El paquete `bio3d` contiene funciones para la lectura, escritura y manipulación de archivos PDB.

Leer un fichero PDB

1. Utiliza la función `read.pdb()` para leer la información sobre el ID '4q21' (https://www.rcsb.org/structure/4q21). Esta función requiere como argumento la especificación del archivo que se va a leer, que puede ser tanto el nombre de un archivo local o el identificador PDB de un archivo del repositorio PDB. ¿De qué clase es el objeto obtenido?

Solución:

```
(pdb <- read.pdb("4q21"))
```

```
##   Note: Accessing on-line PDB file
```

```
##
##  Call:  read.pdb(file = "4q21")
##
##    Total Models#: 1
##      Total Atoms#: 1447,  XYZs#: 4341  Chains#: 1  (values: A)
##
##      Protein Atoms#: 1340  (residues/Calpha atoms#: 168)
##      Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)
##
##      Non-protein/nucleic Atoms#: 107  (residues: 80)
##      Non-protein/nucleic resid values: [ GDP (1), HOH (78), MG (1) ]
##
##    Protein sequence:
##       MTEYKLVVVGAGGVGKSALTIQLIQNHFVDEYDPTIEDSYRKQVVIDGETCLLDILDTAG
##       QEEYSAMRDQYMRTGEGFLCVFAINNTKSFEDIHQYREQIKRVKDSDDVPMVLVGNKCDL
##       AARTVESRQAQDLARSYGIPYIETSAKTRQGVEDAFYTLVREIRQHKL
##
## + attr: atom, xyz, seqres, helix, sheet,
##         calpha, remark, call
```

```
class(pdb)
```

```
## [1] "pdb" "sse"
```

El objeto es de la clase `pdb`. Tiene también una segunda clase, `sse` (*secondary structure elements*), que hace referencia a elementos de estructura secundaria.

---

2. La última parte de la información del objeto `pdb` son los atributos (`+attr`). Es posible obtener los atributos de un objeto utilizando la función `attributes()`,

```
attributes(pdb)
```

```
## $names
## [1] "atom"   "xyz"    "seqres" "helix"  "sheet"  "calpha" "remark" "call"
##
## $class
## [1] "pdb" "sse"
```

Para acceder a cada uno de estos atributos de forma individual, utilizamos la sintaxis común de las listas en R. Obtén la información del atributo átomo o componente.

Solución:

```
head(pdb$atom)
```

```
##   type eleno elety  alt resid chain resno insert      x      y      z o     b
## 1 ATOM     1     N <NA>   MET     A     1   <NA> 64.080 50.529 32.509 1 28.66
## 2 ATOM     2    CA <NA>   MET     A     1   <NA> 64.044 51.615 33.423 1 29.19
## 3 ATOM     3     C <NA>   MET     A     1   <NA> 63.722 52.849 32.671 1 30.27
## 4 ATOM     4     O <NA>   MET     A     1   <NA> 64.359 53.119 31.662 1 34.93
## 5 ATOM     5    CB <NA>   MET     A     1   <NA> 65.373 51.805 34.158 1 28.49
## 6 ATOM     6    CG <NA>   MET     A     1   <NA> 65.122 52.780 35.269 1 32.18
##   segid elesy charge
## 1  <NA>     N   <NA>
## 2  <NA>     C   <NA>
## 3  <NA>     C   <NA>
## 4  <NA>     O   <NA>
## 5  <NA>     C   <NA>
## 6  <NA>     C   <NA>
```

Es un `data.frame`, por lo que podríamos acceder a la distinta información como hacemos habitualmente. Por ejemplo,

```
# Mostrar los dos primeros átomos
pdb$atom[1:2, ]
```

```
##   type eleno elety  alt resid chain resno insert      x      y      z o     b
## 1 ATOM     1     N <NA>   MET     A     1   <NA> 64.080 50.529 32.509 1 28.66
## 2 ATOM     2    CA <NA>   MET     A     1   <NA> 64.044 51.615 33.423 1 29.19
##   segid elesy charge
## 1  <NA>     N   <NA>
## 2  <NA>     C   <NA>
```

```
# Mostrar sólo alguna información de ellos
pdb$atom[1:2, c("eleno", "elety", "x","y","z")]
```

```
##   eleno elety      x      y      z
## 1     1     N 64.080 50.529 32.509
## 2     2    CA 64.044 51.615 33.423
```

Seleccionar átomos

En muchas ocasiones nos interesa seleccionar subconjuntos de átomos que tengan alguna característica de interés. Por ejemplo,

```
(ca.inds <- atom.select(pdb, "calpha"))
```

```
##
##  Call:  atom.select.pdb(pdb = pdb, string = "calpha")
##
##    Atom Indices#: 168  ($atom)
##    XYZ  Indices#: 504  ($xyz)
##
## + attr: atom, xyz, call
```

Esta instrucción selecciona los átomos de carbono α (C-alpha) de la proteína. El C-alpha es el átomo central de cada aminoácido y sirve como referencia estructural porque está presente en todos los residuos y define el esqueleto de la proteína.

La función `atom.select()` devuelve un objeto que contiene los identificadores de los átomos seleccionados (`$atom`) y sus coordenadas tridimensionales (`$xyz`). Esto permite extraer fácilmente las posiciones o propiedades de esos átomos para análisis posteriores. Se pueden consultar otras opciones disponibles con `?atom.select`.

3. El argumento `inverse = TRUE` de la función `atom.select()` permite invertir la selección. Selecciona todos los átomos excepto los átomos de agua (`water`).

Solución:

```
(nowat.inds <- atom.select(pdb, "water", inverse = TRUE))
```

```
##
##  Call:  atom.select.pdb(pdb = pdb, string = "water", inverse = TRUE)
##
##    Atom Indices#: 1369  ($atom)
##    XYZ  Indices#: 4107  ($xyz)
##
## + attr: atom, xyz, call
```

```
head(pdb$atom[nowat.inds$atom, ])
```

```
##   type eleno elety  alt resid chain resno insert      x      y      z o     b
## 1 ATOM     1     N <NA>   MET     A     1   <NA> 64.080 50.529 32.509 1 28.66
## 2 ATOM     2    CA <NA>   MET     A     1   <NA> 64.044 51.615 33.423 1 29.19
## 3 ATOM     3     C <NA>   MET     A     1   <NA> 63.722 52.849 32.671 1 30.27
## 4 ATOM     4     O <NA>   MET     A     1   <NA> 64.359 53.119 31.662 1 34.93
## 5 ATOM     5    CB <NA>   MET     A     1   <NA> 65.373 51.805 34.158 1 28.49
## 6 ATOM     6    CG <NA>   MET     A     1   <NA> 65.122 52.780 35.269 1 32.18
##   segid elesy charge
## 1  <NA>     N   <NA>
## 2  <NA>     C   <NA>
## 3  <NA>     C   <NA>
## 4  <NA>     O   <NA>
## 5  <NA>     C   <NA>
## 6  <NA>     C   <NA>
```

---

4. El argumento operador de `atom.select()` permite combinar varios criterios de selección. Podemos usar `operator="AND"` para que se cumplan todos los criterios a la vez, o `operator="OR"` para que se cumpla al menos uno de ellos. Selecciona todos los átomos que pertenezcan a la proteína (`string = "protein"`) o bien que pertenezcan a un ligando llamado GDP (`resid = "GDP"`).

Solución:

```
(sele <- atom.select(pdb, string="protein", resid="GDP", operator="OR"))
```

```
##
##  Call:  atom.select.pdb(pdb = pdb, string = "protein", resid = "GDP",
##     operator = "OR")
##
##    Atom Indices#: 1368  ($atom)
##    XYZ  Indices#: 4104  ($xyz)
##
## + attr: atom, xyz, call
```

Podemos ver cuántos átomos hay por tipo de residuo con `table()`:

```
table(pdb$atom$resid[sele$atom])
```

```
##
## ALA ARG ASN ASP CYS GDP GLN GLU GLY HIS ILE LEU LYS MET PHE PRO SER THR TYR VAL
##  55 122  32 112  18  28  99 117  44  30  88  96  81  32  55  21  48  77 108 105
```

---

5. Repite la selección utilizando la función `combine.select()`, que permite combinar dos objetos de selección (por ejemplo, dos resultados de `atom.select()`) mediante operadores lógicos como `AND`, `OR`, o `NOT` (argumento `operator`).

Solución:

Necesitamos crear selecciones individuales previamente y posteriormente combinarlas con `combine.select()`, utilizando el argumento `operator`.

```
a.inds <- atom.select(pdb, "protein")
b.inds <- atom.select(pdb, resid="GDP")
(sele <- combine.select(a.inds, b.inds, operator="OR"))
```

```
##  Union of selects
##  *  Selected a total of: 1368 atoms  *
```

```
##
##  Call:  combine.select(sel1 = a.inds, sel2 = b.inds, operator = "OR")
##
##    Atom Indices#: 1368  ($atom)
##    XYZ  Indices#: 4104  ($xyz)
##
## + attr: atom, xyz, call
```

Notad que obtenemos el número de átomos seleccionados.

Trabajar con múltiples ficheros PDB

La función `cat.pdb()` se puede utilizar para concatenar dos o más archivos PDB. Esta función contiene argumentos adicionales para reasignar números de residuos e identificadores de cadena. En el ejemplo a continuación, ilustramos cómo concatenar '4q21' con componentes específicos de '4lhy' (https://www.rcsb.org/structure/4LHY) en un nuevo objeto PDB,

```
# leer los dos pdb
a <- read.pdb("4q21")
```

```
##   Note: Accessing on-line PDB file
```

```
b <- read.pdb("4lhy")
```

```
##   Note: Accessing on-line PDB file
```

```
## trim.pdb() crea un nuevo objeto PDB basado en una seleccion
# cadena A de 4q21
a1 <- trim.pdb(a, chain="A")

# cadenas A, E y F de 4lhy
b1 <- trim.pdb(b, chain="A")
b2 <- trim.pdb(b, chain="E")
b3 <- trim.pdb(b, chain="F")

# creamos un nuevo objeto pdb uniendo esas estructuras
new <- cat.pdb(a1, b1, b2, b3, rechain=TRUE)
# rechain para asignar nuevos identificadores de cadena
unique(new$atom$chain)
```

```
## [1] "A" "B" "C" "D"
```

```
# write.pdb() para escribir un fichero PDB
write.pdb(new, file="4Q21-4LHY.pdb")
```

Visualizaciones 3D de proteínas

Cuando estamos investigando una proteina es importante poder visualizarla en tres dimensiones para tareas como el diseño de fármacos o el modelado de proteínas. En R podemos visualizar estas estructuras en gráficos interactivos utilizando el paquete `NGLVieweR` disponible en CRAN. La información que se maneja es la contenida en la base de datos PDB.

0. Instala y carga la librería.

Solución:

```
if(!("NGLVieweR" %in% installed.packages()))
  install.packages("NGLVieweR")
library("NGLVieweR")
```

Vamos a ilustrar el uso del paquete `NGLVieweR` utilizando como ejemplo el dominio motor de la quinesina humana, cuyo identificador en el PDB es '1BG2'.

1. Para cargar la información de una proteína y visualizarla en 3D, utilizamos la función `NGLVieweR()`, indicando como argumento el ID PDB de la proteína (en este caso, "1BG2").

Añadiendo `addRepresentation("cartoon")`, obtenemos una representación esquemática básica de la estructura.

Podemos encadenar ambas instrucciones usando el operador `%>%` para hacerlo de forma más compacta:

```
NGLVieweR("1BG2") %>%
addRepresentation("cartoon")
```

Your browser/graphics card does not seem to support WebGL.

Find out how to get it here.

Esta instrucción (1) Carga la proteína "1BG2" desde PDB; y (2) la representa en forma de "cartoon", una vista simplificada que muestra la estructura secundaria (hélices, láminas, etc.).

El gráfico que se genera es completamente interactivo. Podemos girar y mover la proteína con el botón izquierdo del ratón, alejar o acercar usando la rueda, y seleccionar regiones de la estructura con un clic.

2. A la función `NGLVieweR()` le pasamos como argumento el identificador PDB de una proteína o un fichero en formato pdb. Ahora, representa la estructura que acabamos de crear y guardar previamente en el archivo local `4Q21-4LHY.pdb`.

Solución:

```
NGLVieweR("4Q21-4LHY.pdb") %>%
addRepresentation("cartoon")
```

Esto te permite visualizar archivos estructurales generados o modificados por ti mismo.

Aunque este paquete se ha desarrollado y testado para ficheros en formatos pdb también soporta otros formatos como .mmcif, .cif, .mcif, .ent, .pqr, .gro, .sdf, .sd, .mol2 and .mmtf.

Tipos de representación y combinaciones

La estructura 3D puede ser representada como "cartoon", "ball+stick", "line", "surface" o "ribbon". Podéis consultar las representaciones disponibles en el manual NGL.js en el epígrafe *Molecular Representations*.

3. Es posible utilizar varios tipos de representación en un mismo gráfico, utilizando la función `addRepresentation()` tantas veces como queráis. Representa la proteína 1BG2 superponiendo dos representaciones distintas.

Solución: Por ejemplo, "cartoon" y "ball+stick":

```
NGLVieweR("1BG2") %>%
addRepresentation("cartoon") %>%
addRepresentation("ball+stick")
```

---

4. También se le puede añadir una animación con las funciones `setRock()` o `setSpin()`. ¿Qué diferencia hay entre estas dos opciones?

Solución:

`setSpin()` para un giro completo alrededor de un eje. Con `setRock()` el giro no es completo.

```
NGLVieweR("1BG2") %>%
  addRepresentation("cartoon") %>%
  addRepresentation("ball+stick") %>%
  setSpin()
```

Notad que estas funciones se concatenan con el operador `%>%`.

Personalización visual por colores y regiones

5. Otro argumento que se puede utilizar de la función `NGLVieweR::addRepresentation()` es `param`. Con este parámetro podemos cambiar el aspecto de algunos residuos. Debemos pasarle una lista especificando las características que nos interese modificar.

5.1. Utiliza `colorScheme = "residueindex"` para colorear la proteína según el índice de residuo.

Solución:

```
# cambiamos colores con el parametro colorScheme
NGLVieweR("1BG2") %>%
  addRepresentation("cartoon",
    param = list(colorScheme = "residueindex"))
```

Hay distintos esquemas disponibles que podeis consultar en [NGL.js] (https://nglviewer.org/ngl/api/manual/coloring.html)

Podemos seleccionar y representar regiones concretas. Por ejemplo para representar los residuos entre las posiciones 233-248 podemos utilizar el elemento `sele` de la lista `param`. Añadiendo elementos a la lista podemos modificar distintos aspectos, por ejemplo el color,

```
# seleccionamos un grupo de AA con sele
NGLVieweR("1BG2") %>%
   addRepresentation("ball+stick",
    param = list(sele = "233-248",
                 colorValue = "yellow",
                 colorScheme = "element"))
```

5.2. Representa la proteina completa en forma de "cartoon" y superpón este segmento en forma de "ball+stick".

Solución:

```
# superponemos representaciones
NGLVieweR("1BG2") %>%
  addRepresentation("cartoon",
    param = list(colorScheme = "residueindex")) %>%
  addRepresentation("ball+stick",
    param = list(sele = "233-248",
                 colorValue = "yellow",
                 colorScheme = "element"))
```

Etiquetado de residuos

Es posible etiquetar aminoácidos utilizando la función `addRepresentation()` con el parámetro `label`. Utilizamos `sele` para seleccionar un aminoácido con la segunda llamada `addRepresentation()`. La tercera llamada sirve para etiquetar ese aminoácido.

```
NGLVieweR("1BG2") %>%
  addRepresentation("cartoon") %>%
  addRepresentation("ball+stick",
                    param = list(colorScheme = "element",
                                 colorValue = "yellow",
                                 sele = "20")) %>%
  addRepresentation("label",param = list(sele = "20",
                                     labelType = "format",
                                     labelFormat = "[%(resname)s]%(resno)s",
                                     labelGrouping = "residue",
                                     color = "white", fontFamiliy = "sans-serif",
                                     xOffset = 1, yOffset = 0, zOffset = 0,
                                     fixedSize = TRUE, radiusType = 1, radiusSize = 1.5))
```

Esquematizar la información disponible en *UniProt* de una proteína

Otra base de datos sobre proteínas que solemos manejar es *UniProt*. Esta base de datos proporciona información detallada sobre secuencias de proteínas, anotaciones funcionales, estructuras, expresión, interacciones, etc, centrandose en la secuencia y función de las proteínas más que en la estructura tridimensional como *PDB*. Desde R podemos utilizar la información disponible en *UniProt* para crear esquemas sobre una proteína utilizando el paquete `drawProteins` de Biconductor.

0. Instala y carga el paquete.

Solución:

```
if(!("drawProteins" %in% installed.packages()))
  BiocManager::install("drawProteins")
library("drawProteins")
```

---

1. Utiliza la función `get_features()` para recuperar información estructural y funcional de la proteína: secuencia, dominios, regiones desordenadas, motivos, sitios de fosforilación, etc.

Almacena en un objeto la información obtenida con `get_features()` para la proteína Rel A o NF-kappaB, p65 que es un factor de transcripción de DNA muy conocido y cuyo identificador es "Q04206". Podéis ver la información disponible en *UniProt* en el enlace (https://www.uniprot.org/uniprot/Q04206). ¿Que tipo de objeto se obtiene como resultado?

Solución:

```
rela <- drawProteins::get_features("Q04206")
```

```
## [1] "Download has worked"
```

```
class(rela)
```

```
## [1] "list"
```

Se obtiene un objeto de clase `list` que contiene la anotación completa de la proteína.

---

2. Para trabajar más fácilmente con los datos, convierte el objeto anterior en un `data.frame` con `feature_to_dataframe()`.

Solución:

```
rela.data <- drawProteins::feature_to_dataframe(rela)
head(rela.data)
```

```
##                  type                      description begin end length
## featuresTemp    CHAIN           Transcription factor p65     1 551    550
## featuresTemp.1 DOMAIN                              RHD    19 306    287
## featuresTemp.2 REGION                        Disordered   309 348     39
## featuresTemp.3 REGION Transcriptional activation domain 3   342 389     47
## featuresTemp.4 REGION Transcriptional activation domain 1   415 476     61
## featuresTemp.5 REGION                        Disordered   506 530     24
##                accession  entryName taxid order
## featuresTemp      Q04206 TF65_HUMAN  9606     1
## featuresTemp.1    Q04206 TF65_HUMAN  9606     1
## featuresTemp.2    Q04206 TF65_HUMAN  9606     1
## featuresTemp.3    Q04206 TF65_HUMAN  9606     1
## featuresTemp.4    Q04206 TF65_HUMAN  9606     1
## featuresTemp.5    Q04206 TF65_HUMAN  9606     1
```

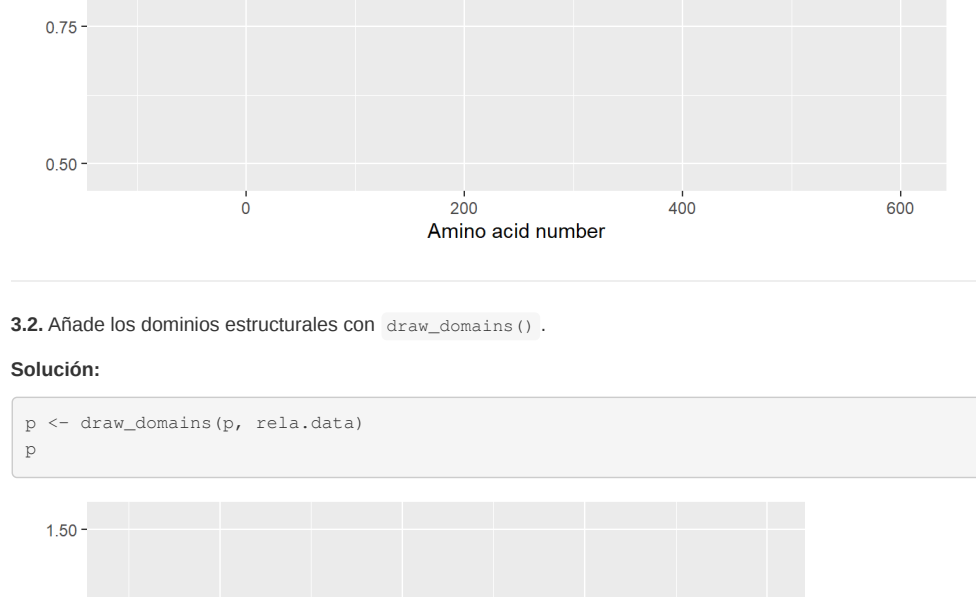
---

3. Vamos a esquematizar la información de este proteína utilizando el paquete `ggplot2`.

Primero dibujamos el área proporcional a la longitud de la proteína usando `draw_canvas()`. Utiliza esta función con argumento el `data.frame` que has creado en el paso anterior. Almacena el resultado en un objeto `ggplot` y muestra el gráfico resultante.

Solución:

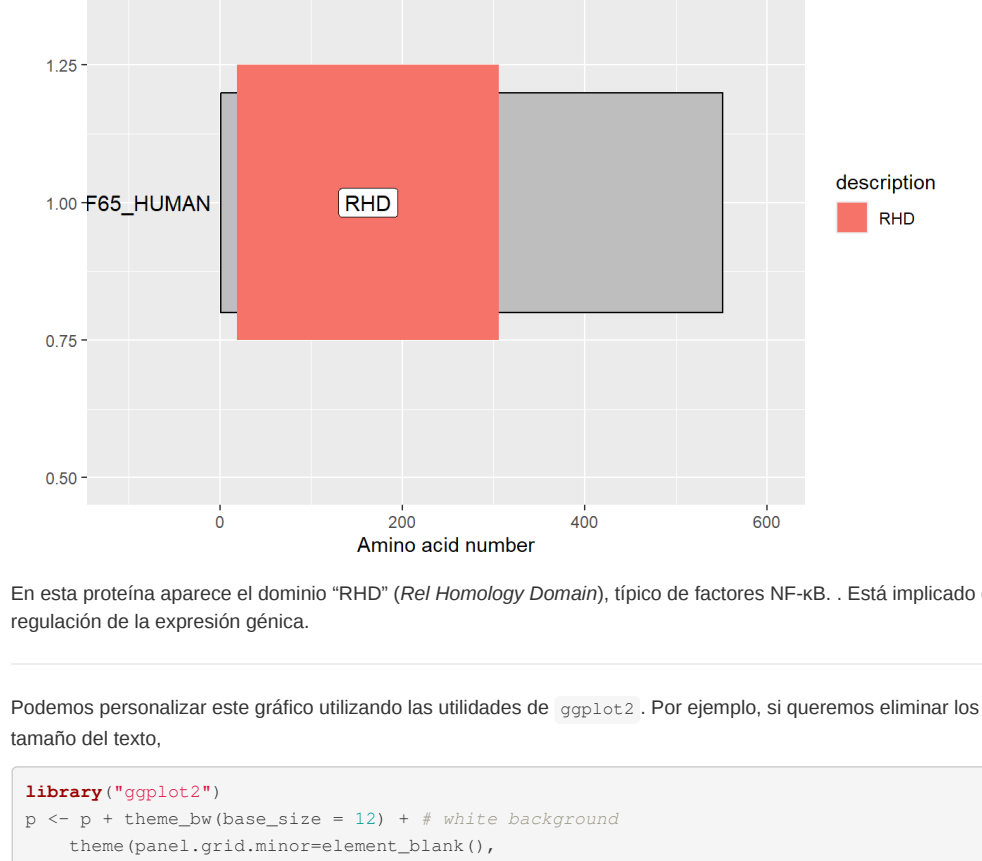
```
p <- draw_canvas(rela.data)
p
```



3.1. Dibuja la cadena principal de la proteína con `draw_chains()`. Esta función requiere como primer argumento el objeto `ggplot` creado en el paso anterior y el `data.frame` con la información.

Solución:

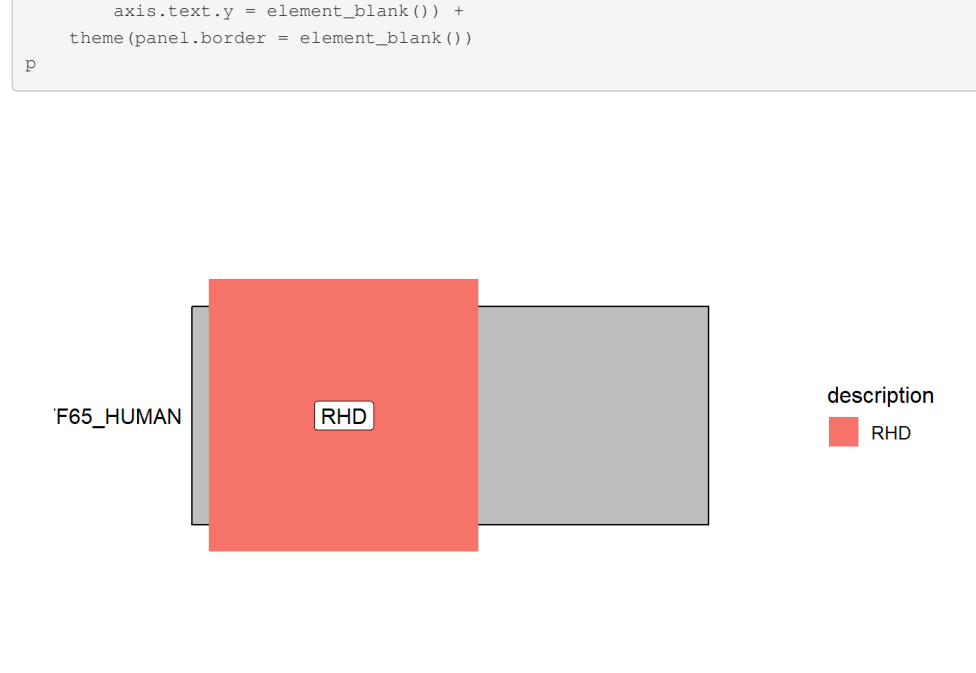
```
p <- draw_chains(p, rela.data)
p
```



3.2. Añade los dominios estructurales con `draw_domains()`.

Solución:

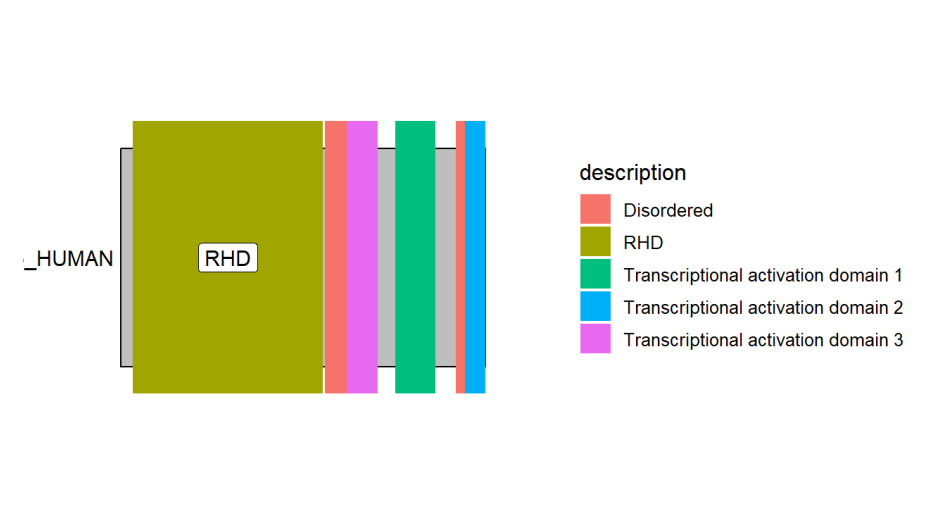
```
p <- draw_domains(p, rela.data)
p
```



En esta proteína aparece el dominio "RHD" (*Rel Homology Domain*), típico de factores NF-κB. . Está implicado en la unión al ADN y en la regulación de la expresión génica.

Podemos personalizar este gráfico utilizando las utilidades de `ggplot2`. Por ejemplo, si queremos eliminar los ejes, el fondo y controlar el tamaño del texto,

```
library("ggplot2")
p <- p + theme_bw(base_size = 12) + # white background
    theme(panel.grid.minor=element_blank(),
        panel.grid.major=element_blank()) +
    theme(axis.ticks = element_blank(),
        axis.text.y = element_blank()) +
    theme(panel.border = element_blank())
p
```



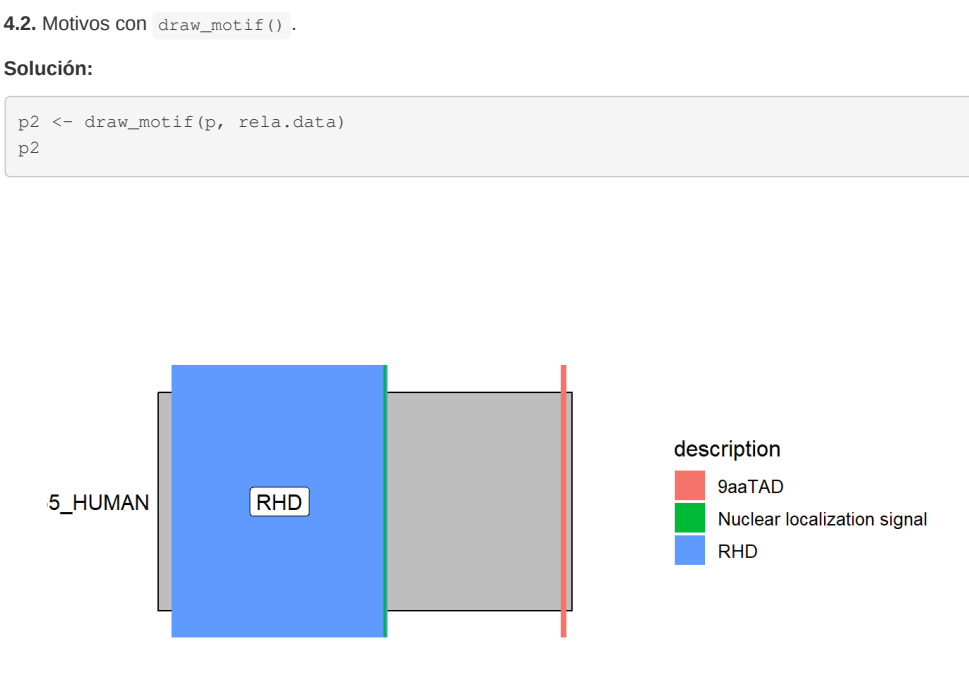
Añadir otras características

4. Hemos creado la estructura básica, pero podemos añadir muchas otras características de interés.

4.1. Regiones de interés con `draw_regions()`.

Solución:

```
p1 <- draw_regions(p, rela.data)
p1
```



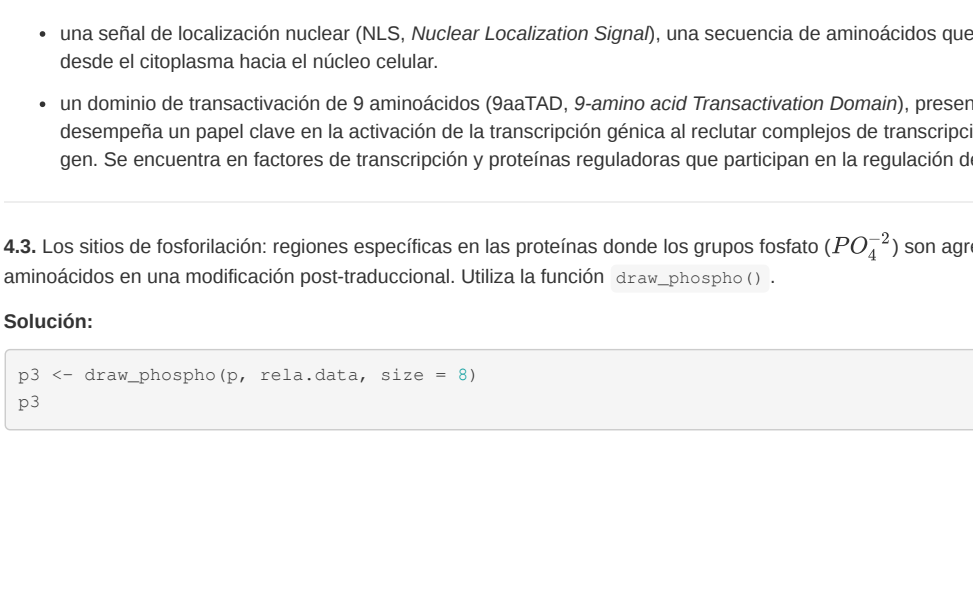
Al dominio "RHD" se han añadido:

- Regiones desordenadas (sin estructura terciaria estable).
- Dominios de activación transcripcional, implicados en la activación génica mediante el reclutamiento de complejos transcripcionales.

4.2. Motivos con `draw_motif()`.

Solución:

```
p2 <- draw_motif(p, rela.data)
p2
```



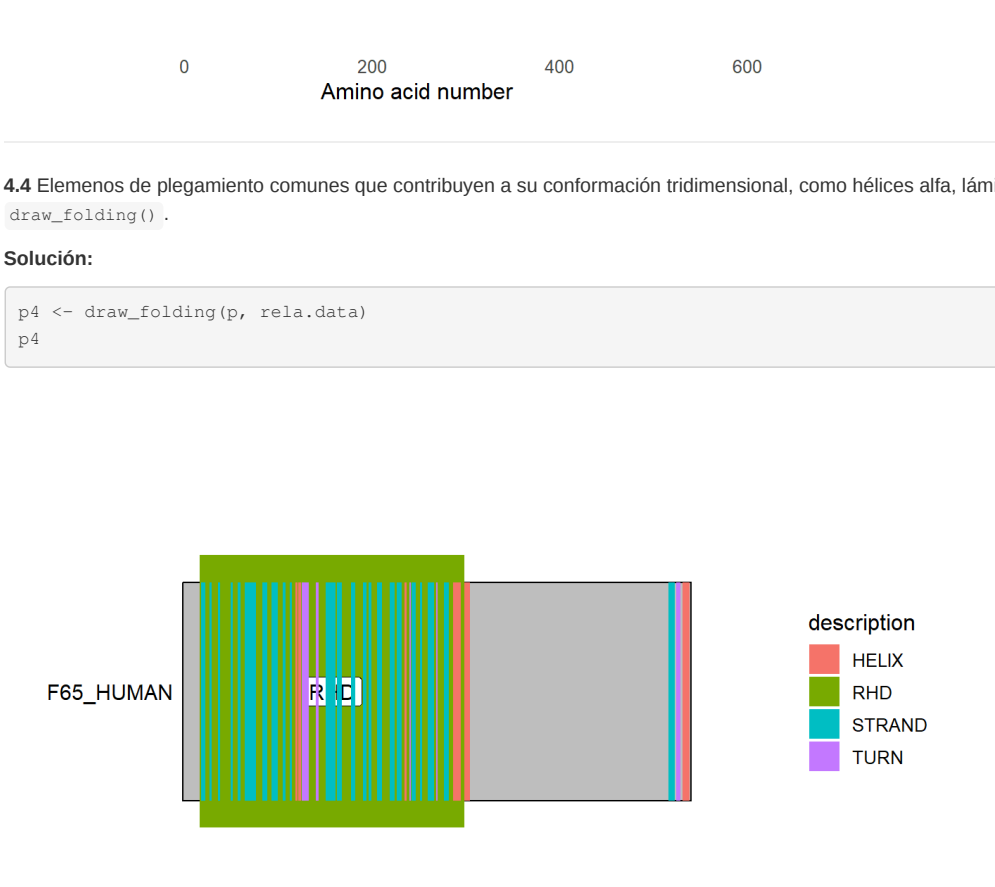
En este caso se añade,

- una señal de localización nuclear (NLS, *Nuclear Localization Signal*), una secuencia de aminoácidos que facilita su transporte selectivo desde el citoplasma hacia el núcleo celular.
- un dominio de transactivación de 9 aminoácidos (9aaTAD, *9-amino acid Transactivation Domain*), presente en proteínas eucariotas que desempeña un papel clave en la activación de la transcripción génica al reclutar complejos de transcripción y coactivadores al promotor del gen. Se encuentra en factores de transcripción y proteínas reguladoras que participan en la regulación de la expresión génica.

4.3. Los sitios de fosforilación: regiones específicas en las proteínas donde los grupos fosfato (PO_4^{-2}) son agregados covalentemente a los aminoácidos en una modificación post-traduccional. Utiliza la función `draw_phospho()`.

Solución:

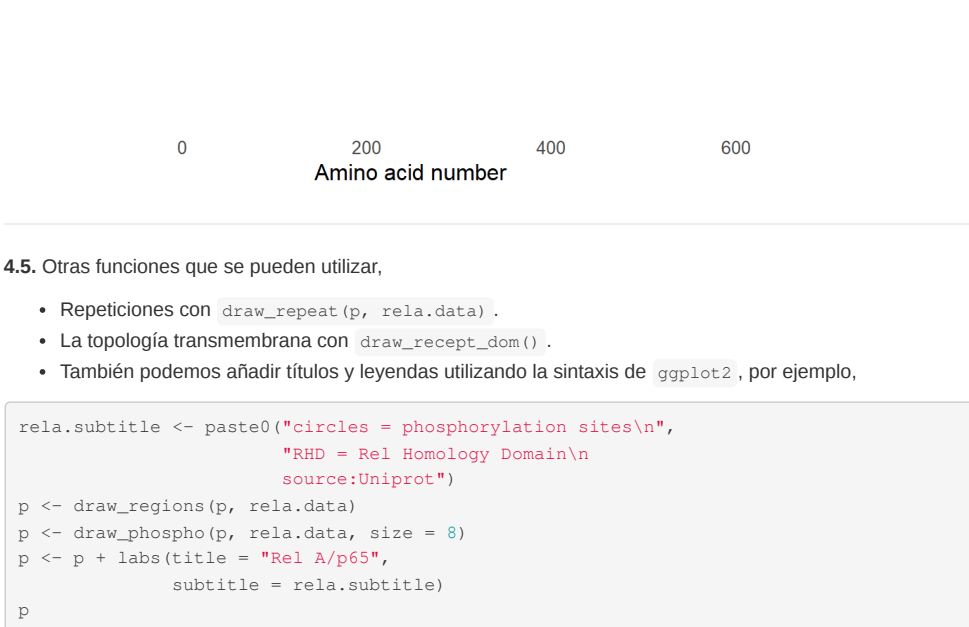
```
p3 <- draw_phospho(p, rela.data, size = 8)
p3
```



4.4 Elemenos de plegamiento comunes que contribuyen a su conformación tridimensional, como hélices alfa, láminas beta y giros, con `draw_folding()`.

Solución:

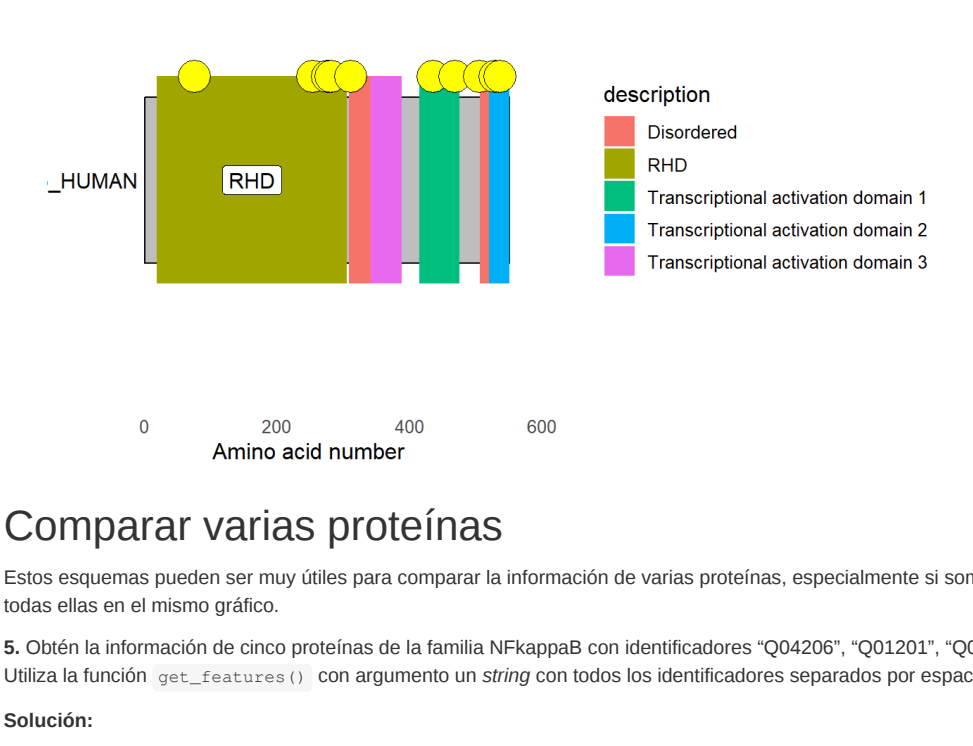
```
p4 <- draw_folding(p, rela.data)
p4
```



4.5. Otras funciones que se pueden utilizar,

- Repeticiones con `draw_repeat(p, rela.data)`.
- La topología transmembrana con `draw_recept_dom()`.
- También podemos añadir títulos y leyendas utilizando la sintaxis de `ggplot2`, por ejemplo,

```
rela.subtitle <- paste0("circles = phosphorylation sites\n",
                        "RHD = Rel Homology Domain\n
                        source:Uniprot")
p <- draw_regions(p, rela.data)
p <- draw_phospho(p, rela.data, size = 8)
p <- p + labs(title = "Rel A/p65",
              subtitle = rela.subtitle)
p
```



Comparar varias proteínas

Estos esquemas pueden ser muy útiles para comparar la información de varias proteínas, especialmente si somos capaces de esquematizar todas ellas en el mismo gráfico.

5. Obtén la información de cinco proteínas de la familia NFkappaB con identificadores "Q04206", "Q01201", "Q04864", "P19838" y "Q00653". Utiliza la función `get_features()` con argumento un *string* con todos los identificadores separados por espacios.

Solución:

```
prot.data <- drawProteins::get_features("Q04206 Q01201 Q04864 P19838 Q00653")
```

```
## [1] "Download has worked"
```

```
prot.data <- drawProteins::feature_to_dataframe(prot.data) #como data.frame
```

---

6. Construye el gráfico,

Solución:

El código que necesitamos para hacer este gráfico es,

```
p <- draw_canvas(prot.data)
p <- draw_chains(p, prot.data)
p <- draw_domains(p, prot.data)
p <- draw_repeat(p, prot.data)
p <- draw_motif(p, prot.data)
p <- draw_phospho(p, prot.data, size = 8)

# background and y-axis
p <- p + theme_bw(base_size = 14) + # white backgnd & change text size
    theme(panel.grid.minor=element_blank(),
        panel.grid.major=element_blank()) +
    theme(axis.ticks = element_blank(),
        axis.text.y = element_blank()) +
    theme(panel.border = element_blank())

# add titles
rela.subtitle <- paste0("circles = phosphorylation sites\n",
                        "RHD = Rel Homology Domain\nsource:Uniprot")

p <- p + labs(title = "Schematic of human NF-kappaB proteins",
              subtitle = rela.subtitle)


# move legend to top
p <- p + theme(legend.position="top") + labs(fill="")
p
```